Radix Sort

Introduction

Radix Sort is a non-comparative sorting algorithm that sorts numbers by processing their digits one place at a time.
Instead of comparing elements directly (like in Bubble Sort or Quick Sort), it groups numbers based on each digit's place value — such as ones, tens, hundreds, etc.

Radix Sort uses another sorting algorithm, usually Counting Sort, as a subroutine to sort elements by each digit.

Concept

Radix Sort works by sorting the input numbers digit by digit, starting either from:

- The least significant digit (LSD), rightmost digit, or
- The most significant digit (MSD) is, leftmost digit.

In this topic, we focus on LSD Radix Sort, which starts from the ones place and moves left.

The process repeats for every digit position until all digits have been considered.
At each step, the algorithm groups elements into buckets (0–9) based on their current digit and then collects them back in order.

Step-by-Step Procedure

Step 1: Identify the largest number

- Determine the number with the most digits in the list.
- This tells us how many rounds of sorting we'll need.
  Example:
  If the largest number is `802`, we'll sort based on the 1's, 10's, and 100's places.

Step 2: Start with the least significant digit (1's place)

- Group all numbers based on their 1's digit.
- For example, if the list is `[170, 45, 75, 90, 802, 24, 2, 66]`:
    - 170 → bucket 0
    - 45 → bucket 5
    - 75 → bucket 5
    - 90 → bucket 0
    - 802 → bucket 2
    - 24 → bucket 4
    - 2 → bucket 2
    - 66 → bucket 6
- Then, collect them back in bucket order: `[170, 90, 802, 2, 24, 45, 75, 66]`

Step 3: Move to the next digit (10's place)

- Sort the new list based on the 10's digit:
    - 170 → 7
    - 90 → 9
    - 802 → 0
    - 2 → 0
    - 24 → 2

    - 45 → 4
    - 75 → 7
    - 66 → 6
- After grouping and collecting: `[802, 2, 24, 45, 66, 170, 75, 90]`

Step 4: Move to the next digit (100's place)

- Sort based on the 100's digit:
    - 802 → 8
    - 2 → 0
    - 24 → 0
    - 45 → 0
    - 66 → 0
    - 170 → 1
    - 75 → 0
    - 90 → 0
- Final sorted order: `[2, 24, 45, 66, 75, 90, 170, 802]`

Step 5: Repeat until all digits have been processed

Once all digits (from least to most significant) are sorted, the entire list will be sorted in ascending order.

Code Implementation

```
def counting_sort(arr, place):
```

```
    n = len(arr)
    output = [0] * n
    count = [0] * 10

    for i in range(n):
        digit = (arr[i] // place) % 10
        count[digit] += 1

    for i in range(1, 10):
        count[i] += count[i - 1]

    i = n - 1
    while i >= 0:
        digit = (arr[i] // place) % 10
        output[count[digit] - 1] = arr[i]
        count[digit] -= 1
        i -= 1

    for i in range(n):
        arr[i] = output[i]

def radix_sort(arr):
    max_num = max(arr)
    place = 1
    while max_num // place > 0:
        counting_sort(arr, place)
        place *= 10
```

Complexity Analysis

Aspect	Description
Time Complexity	O(nk), where n is the number of elements and k is the number of digits in the largest element.
Space Complexity	O(n + b), where b is the base (10 for decimal digits).
Stability	Stable – maintains the order of equal elements.
Type	Non-comparative sorting algorithm.

Advantages

- Very efficient when sorting numbers or strings of fixed length.
- Faster than comparison-based algorithms for small-digit or bounded values.
- Stable, so suitable when the order of equal elements matters.

Limitations

- Works best for numbers with limited digits.
- Requires extra space for buckets and intermediate arrays.
- Not ideal for very large or variable-length data.

Use Cases

- Sorting:
    - Student IDs
    - Phone numbers
    - Zip codes
    - Serial or account numbers
- When dealing with integers or strings of fixed length.